

AI SecOps Multi-Agent Pipeline — Node / State / Function Flow

LangGraph StateGraph(SOCState) | app.invoke({raw_log, log_type})

SOCState (공유 상태)

입력: raw_log, log_type → 중간: parsed_log, classification, red_team, blue_team → 출력: verdict, mitre, incident_report

START
app.invoke({raw_log, log_type})

① parse_node

☒ State 읽기: raw_log, log_type

parse_log(raw_log, log_type)

↳ 반환 → State: {"parsed_log": ParsedLog}

필드: ParsedLog: log_type, raw, normalized, iocs[]

② classify_node

☒ State 읽기: parsed_log

ask_json(model, ..., schema=ClassificationResult)

↳ 반환 → State: {"classification": ClassificationResult}

필드: is_attack, confidence, attack_type, summary

③ red_team_node

☒ parsed_log, classification

ask_json(..., schema=RedTeamAnalysis)

↳ {"red_team": RedTeamAnalysis}

attack_hypothesis, evidence[], confidence

병렬 분기 (Parallelization)

③ blue_team_node

☒ parsed_log, classification

ask_json(..., schema=BlueTeamAnalysis)

↳ {"blue_team": BlueTeamAnalysis}

↳ 반환 → State: {"verdict": JudgeVerdict}

design, hypothesis, evidence[], confidence

④ judge_node

☒ State 읽기: parsed_log, classification, red_team, blue_team

ask_json(..., schema=JudgeVerdict)

필드: verdict, tp_fp, confidence, rationale, final_rationale

⑤ mitre_node

☒ State 읽기: parsed_log, verdict

ask_json(..., schema=MitreMapping)

↳ 반환 → State: {"mitre": MitreMapping}

필드: technique_ids[], recommendations[], severity

⑥ report_node

☒ State 읽기: parsed_log, verdict, mitre

ask_json(..., schema=IncidentReport)

↳ 반환 → State: {"incident_report": IncidentReport}

필드: title, executive_summary, full_report_markdown

END → result (전체 SOCState)

Function Call Detail — parse_log & ask_json 내부 흐름

parse_log(raw_log, log_type) → ParsedLog

- raw_log가 dict → json.dumps / str → raw 문자열
- log_type=='auto' → process/command_line 키워드로 edr|waf 자동 판별
- IP_RE 정규식 → iocs[] (type=ip)
- uri/url 필드 → iocs[] (type=url)
- return ParsedLog(log_type, raw, normalized, iocs)



ask_json(model, system, user, log_raw, schema) → dict

model is None (mock)	.mock_json(system, user, log_raw) → log_is_attack() 키워드 휴리스틱
schema 있음	model.with_structured_output(schema).invoke(prompt) → Pydantic.model_dump() → dict
fallback	model.invoke(prompt + strict JSON 지시)
응답 파싱	.to_text(response.content) # Gemini list 대응
JSON decode	.parse_json_text: json → json5 → json_repair
재시도	파싱 실패 시 model.invoke() 1회 재요청



get_chat_model() → ChatModel | None

- LLM_PROVIDER=mock → None | gemini → init_chat_model(GEMINI, google_genai) | openai → init_chat_model(OPENAI)
- 노드에서 model / _llm() 호출 → ask_json에 전달



LangGraph State 병합 규칙

- 각 node 함수: state: SOCState 읽기 → dict 반환 → LangGraph가 키별 merge
- 예: classify_node returns {classification: ...} → state[classification] 갱신
- 병렬 red_team + blue_team 각각 독립 dict 반환 → judge 진입 전까지 두 필드 모두 채워짐
- 최종 app.invoke() 반환값 = 모든 필드가 채워진 SOCState dict

Graph Edges (workflow.compile())

- START → parse → classify
- classify → red_team || classify → blue_team (병렬)
- red_team → judge + blue_team → judge (합류)
- judge → mitre → report → END

Pydantic Models (Agent 간 데이터 형식)

ParsedLog | ClassificationResult | RedTeamAnalysis | BlueTeamAnalysis | JudgeVerdict | MitreMapping | IncidentReport | IOC

실행 예: result = app.invoke({'raw_log': 'WAF_LOG', 'log_type': 'waf'})

result['verdict'].final_rationale | result['incident_report'].executive_summary